Renting Compute

Lesson 2 of 12 · Which chip, how to pay, and how to stop the meter.

In this lesson → ① Four chip families ② Reading the names ③ The buying ladder ④ The commitment trap ⑤ Spot and checkpoints ⑥ Price per petaflop ⑦ Scale to zero
Course project: ShopNest Copilot, the AI support assistant we move to the cloud, step by step.

1 Renting compute: three questions

Which **chip**? How to **pay**? When to **stop**? Prices and chip names are vendor figures dated **14 Oct 2026**. The sums and simulations are mine and labelled.

2 Four families of chips



same job, four brands, four software stacks

Family	Software	Where
NVIDIA	CUDA	every cloud
AMD (MI35SX)	ROCm	OCI, Vultr (GA)
Google TPU	JAX / XLA	Google Cloud only
AWS Trainium / Inferentia	Neuron SDK	AWS only

Rule
The more special the chip, the cheaper it can be, and the more you depend on one cloud. Trade on purpose.

3 Chips to know (Oct 2026)

Chip	Status	Memory
H100 / H200	workhorse, GA	80 / 141 GB
B200	GA	192 GB
B300 / GB300	broadly rentable	288 GB
Vera Rubin	**limited** (one cloud, 30 Sep 2026)	288 GB
MI355X (AMD)	GA on OCI, Vultr	288 GB
TPU Ironwood	GA Apr 2026	192 GB, pod of 9,216
Trainium3	GA Dec 2025	144 GB, UltraServer of 144

Vendor figures, mostly from secondary sources for the newest chips: treat as approximate.

4 Four status words

- ✓ **GA**: rent it today
- ✓ **limited**: a waiting list
- ✓ **announced**: only a slide, never plan on it
- ✓ **old**: often the best value

The real cost of switching away from NVIDIA

→ Stock PyTorch / JAX models move fairly easily
→ Custom CUDA kernels need engineering time
→ Test **your own workload** before you commit

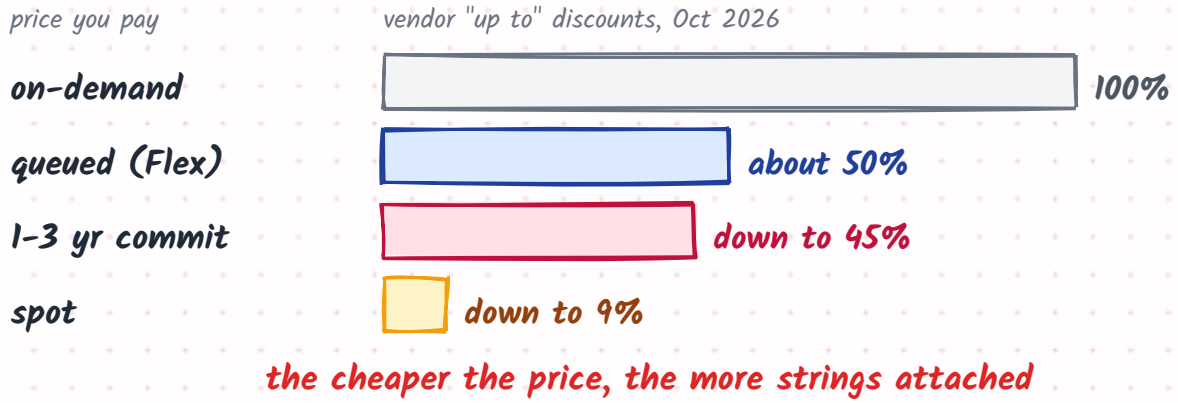
5 Reading machine names

Cloud	Name	Means
AWS	p5.48xlarge	8 × H100, the whole host
Google	a3-highgpu-8g	8 GPUs (the 8g)
Azure	Standard_ND96isr_H100_v5	8 × H100; i isolated, s premium storage, r RDMA

6 Big clouds and neoclouds

	Neocloud	Big cloud
List price	about ½ to ⅔ of the big cloud	higher
New chips	often first	later
Strengths	GPU price	your data, identity, network, managed services, own chips

7 The buying ladder

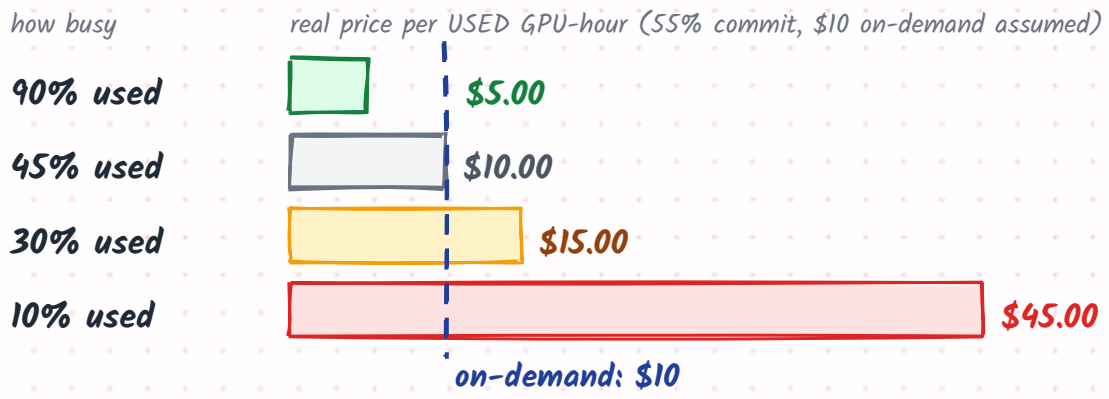


the cheaper the price, the more strings attached

Vendor "up to" discounts: commitments up to ~55% off, queued (Flex) ~50% off for some chips, spot up to 91% off. Real discounts vary.

8 The commitment trap

> **real price per used hour = rate ÷ how busy you keep it**



Plan (assumed $10 on-demand)	Rate	Busy	Real $ / used hour
on-demand	$10.00	100%	$10.00
55% commit	$4.50	90%	**$5.00**
55% commit	$4.50	45%	$10.00
55% commit	$4.50	30%	**$15.00**
55% commit	$4.50	10%	**$45.00**

Break-even
A commitment pays off only if busy > **1 − discount**. 55% off needs more than **45%** use. Commit to the steady part, never the peak.

9 Queued and prepaid capacity

→ **Queued** (e.g. Flex-start): about half price, your job waits for a slot. Good for nightly evals, bad for a customer waiting

→ **Prepaid windows** (e.g. AWS Capacity Blocks): reserve a stretch of time for a deadline

10 Spot

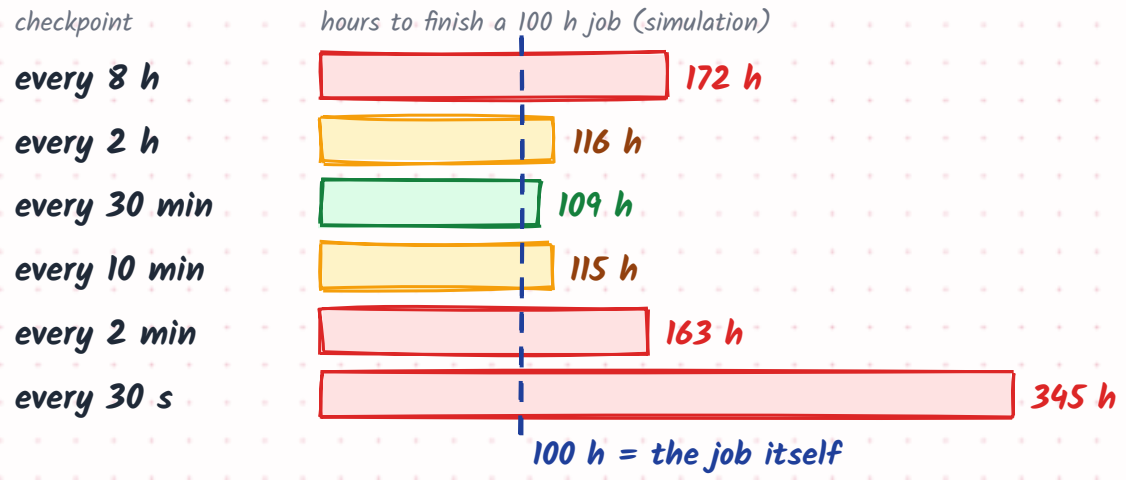
> Spare capacity at up to ~91% off. The provider can take it back: **2 minutes** notice on AWS, **30 seconds** on Azure.

The simulation (not real machines)

Setting	Value
job	100 hours of work
interruptions	every 8 h on average (**assumed**)
restart	0.1 h, and all work since the last save is lost
one save	0.02 h (72 s)
price	spot = 30% of on-demand (**assumed**)
runs	2,000 per setting, seed 42

Script
l02_spot_checkpoint.py *in the code zip*

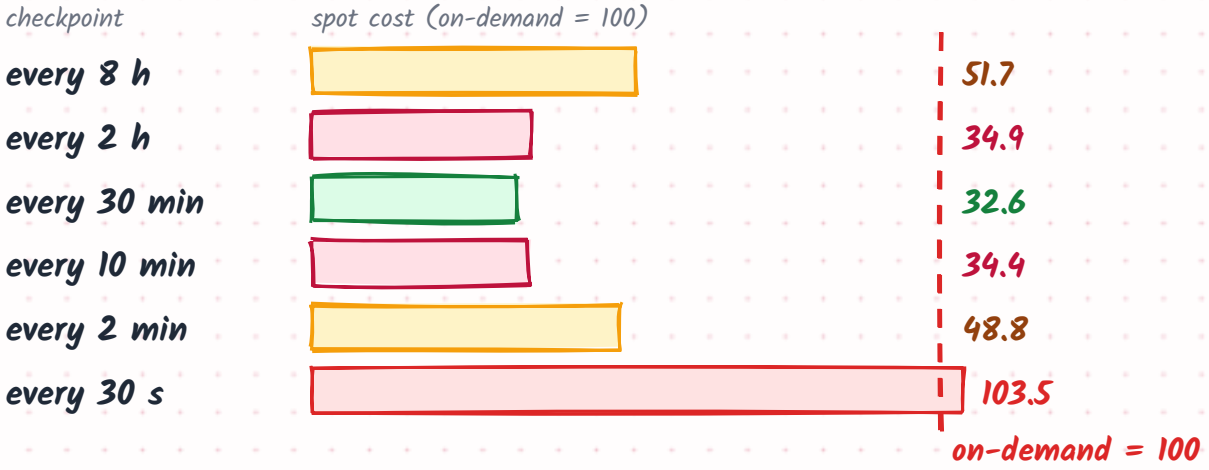
11 How often to save?



Save every	Hours	Spot cost (on-demand = 100)
never (end only)	**2,203,564**	661,069
8 h	172.4	51.7
2 h	116.3	34.9
30 min	**108.8**	**32.6**
34 min (Young)	108.8	32.6
10 min	114.7	34.4
2 min	162.6	48.8
30 s	344.8	**103.5**

12 The sweet spot

> **interval ≈ √(2 × MTBF × cost of one save)** = √(2 × 8 × 0.02) = 0.566 h ≈ **34 min**

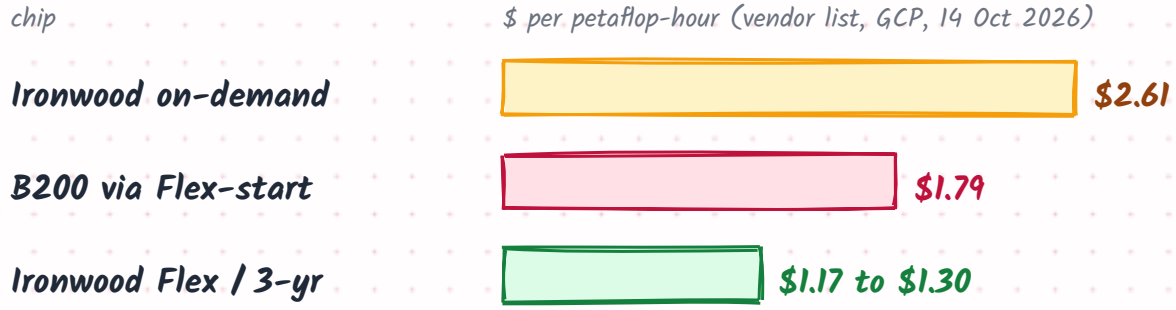


- ✓ Save too rarely: you lose a lot of work each interruption
- ✓ Save too often: you spend your time saving (every 30 s cost **more than on-demand**)
- ✓ Handle the warning notice: save when it arrives

Where spot fits
Training with saves, batch evals, data jobs. Not your only live endpoint: keep some on-demand behind it.

13 Price per petaflop-hour

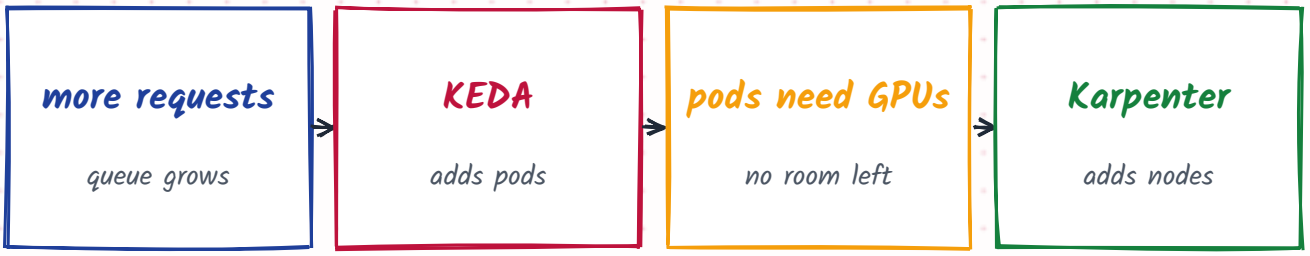
> $ per PF-hour = chip $ per hour ÷ dense FP8 petaflops



Chip and plan	$ / hour	PF	$ / PF-hr
Ironwood on-demand	12.00	4.6	2.61
B200 via Flex-start	8.06	4.5	1.79
Ironwood Flex or 3-yr	5.40 to 6.00	4.6	1.17 to 1.30

Vendor list prices on Google Cloud, 14 Oct 2026. Divide first: the cheapest hourly price is not always the cheapest work.

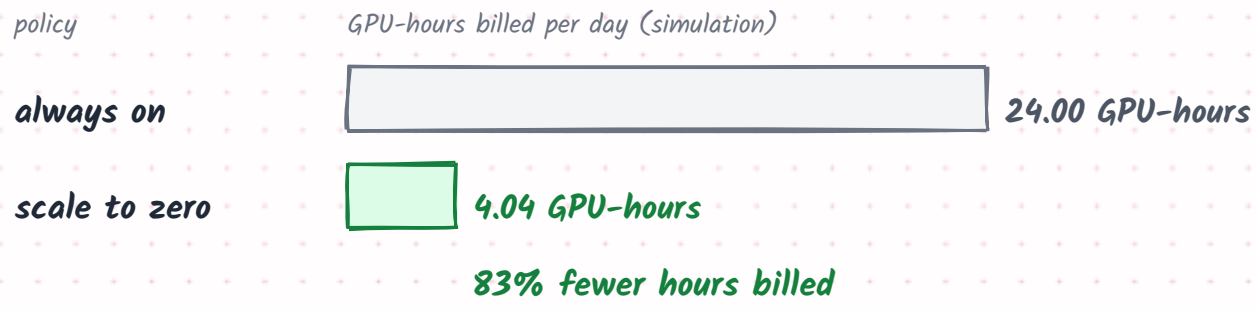
14 Autoscaling: stop the meter



pods first, then nodes, then minutes of waiting

→ Pods = running copies of your service. **KEDA** adds pods when requests queue up
→ No room? **Karpenter** or the Cluster Autoscaler adds machines (minutes for a GPU)
→ Scale on an early signal like **queue length**, not on GPU busy time

15 Scale to zero, simulated



One GPU, 5 s per request. Busy 3 h (a request every 12 s), light 1 h (every 120 s), silent otherwise. Idle timeout 5 min, cold start 60 s (**assumed**). 200 seeded days. Script: l02_scale_to_zero.py.

16 The price of scale to zero

Policy	GPU-hours / day	Mean wait	Slowest 5%	Max wait	Cold starts / day
always on	24.00	1.71 s	7.90 s	39.1 s	0
scale to zero	**4.04**	2.31 s	8.86 s	80.4 s	**3.86** (60 s each)

mean wait

1.7 s to 2.3 s

slowest 5%

7.9 s to 8.9 s

3.9 cold starts a day

60 s each

cheap on average, ugly for the first user

Softening a cold start (taught, not measured here)

→ Cache model files close to the GPU
→ Save compiled artifacts
→ Snapshot a loaded model
→ Or keep one machine warm and scale the rest to zero

17 ShopNest Copilot: three workloads

Workload	Rung of the ladder
live assistant (never sleeps)	commit the steady part, on-demand for peaks
nightly eval run (patient)	spot + save about every 30 min
staging (a few hours a day)	scale to zero

18 Words to keep

Word	Meaning
Commitment	a promise to pay for a discount
Spot	spare capacity you can lose
Checkpoint	a saved copy of your progress
Cold start	the wait when a machine starts from nothing

19 Interview corner

Q. When is a commitment worth it?

A. Only if busy > 1 − discount. At 55% off you need more than 45% use, otherwise the real price per used hour is above on-demand.

Q. How do you use spot safely?

A. Checkpoint regularly, handle the interruption notice, and pick the interval with √(2 × MTBF × save cost). Never run your only live endpoint on spot.

Q. Why can saving too often cost more than on-demand?

A. Each save costs work time. In my simulation, saving every 30 s took 345 hours for a 100 hour job, so spot cost 103.5 against 100 on-demand.

Q. What is the downside of scale to zero?

A. Cold starts. In my simulation 3.9 users a day waited a full 60 s while the GPU hours fell from 24 to 4.04.

20 Your turn

102_spot_checkpoint.py: change the interruption rate and save cost.
102_buying_math.py: your price and busy time.
102_scale_to_zero.py: your traffic.

⚡ Revise in 60 seconds

- → commit pays off only if busy > **1 − discount**
- → no saves: **2,203,564 h** for a 100 h job
- → save every **34 min**: 109 h, cost 32.6 vs 100
- → save every 30 s: cost **103.5** (worse than on-demand)
- → sweet spot = √(2 × MTBF × save cost)
- → scale to zero: **24** → **4.04** GPU-hours / day
- → 3.9 cold starts a day, 60 s each
- → price per PF-hour: divide first

Next lesson → CAI-03
Containers and Kubernetes for AI: package a model so it runs the same everywhere
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**